

Aave Labs - Stable Vaults

Final report

Prepared by: Josselin Feist (josselin@seceureka.com)

July 6th, 2026

Prepared For: Aave Labs

Executive Summary	4
Overview	4
Security findings	6
Dust positions can prevent adding a new subvault	8
Summary	8
Detailed description	8
Recommendations	9
Status	9
allowOutOfOrderExecution = false is deprecated	10
Summary	10
Detailed description	10
Recommendations	11
Status	11
ChainId collision risks on ccipReceive	12
Summary	12
Detailed description	12
Recommendations	13
Status	13
Front-running in applyWithdrawalPolicy	14
Summary	14
Detailed description	14
Recommendations	14
Status	14
Unstable rounding in growth factor	15
Summary	15
Detailed description	15
Status	15
Temporary denial of service through funds move	16
Summary	16
Recommendations	16
Status	16
Cross-chain message with dust might be trapped	17
Summary	17
Detailed description	17
Recommendations	17
Status	18
The earning chain might send an outdated balance	19
Summary	19
Detailed description	19
Reentrancy on rebalance	19
Timing attack on update	19

Recommendations	20
Status	20
ATokenVaultMerkleRewardClaimer's owner can drain the vault	21
Summary	21
Detailed description	21
Recommendations	22
Status	23
Fee subsidy with stuck native currency token	24
Summary	24
Detailed description	24
Recommendations	24
Status	24
Lack of zero-address check on previous signer in transferSigner	25
Summary	25
Recommendations	25
Status	25
Manager can drain dust through migration operations	26
Summary	26
Detailed description	26
Recommendations	27
Status	27
Lack of Sequencer's uptime check can lead to stale prices and balances being used	28
Summary	28
Detailed description	28
Recommendations	29
Status	29
Uncaught adapter reverts can block getChainBalance	30
Summary	30
Recommendations	31
Status	31
Deposit share should be calculated from the net deposited amount	32
Summary	32
Recommendations	32
Status	32
Appendix A: Design recommendations against depegging	33
Solution 1: Liquidity Pool Per Asset Design	34
Approach	34
Example User Flow	34
Caveat	35
Solution 2: Oracle-Based Asset Valuation with Current Architecture	35
Approach	35

Example User Flow	35
Caveat	35
Appendix B: Defense in depth recommendations	36
Appendix C: Rounding analysis	37
Precision loss worst case scenario	37
Reuse problem	38
Splitting strategy to break the rate growth ordering	38

Executive Summary

Overview

From January 12th to February 6th, we reviewed the Stable Vault protocol, working from commit `c5f16f1994740d69608f99e2f472063762816568`.

Our review focused on the Stable Vault/Funds Handler/Gateways interactions and the CCIP binding. We reviewed the risks of balance desynchronization, the IOU mechanism, and the bridges. We also reviewed the Merkl Reward Claiming PR ([PR112](#), commit `551654011b603e00cf259fa601ce712c0990bfcf`). Our review included a best-effort pass on the ERC6900 implementation (`31bbbce8488c41172ab10dd63bf137a990d57c06`), however, this component was deprioritized per Aave Labs' request.

Through the review, we explored solutions to improve the system's resilience against asset depegging, and we proposed two directions, which are summarized in [Appendix A](#). We also proposed general recommendations to improve the contracts in [Appendix B](#). These recommendations are not tied to specific issues and are best practices. We also performed a detailed precision rate analysis of the conversion rate mechanism, and we provided our notes in [Appendix C](#).

Aave Labs decided to integrate oracles into the system to mitigate the risk of assets depegging, and we partially reviewed [PR157](#), which contains the new oracle integration.

Based on the findings from this report, the ones found by Aave Labs, and other security researchers, we recommend that for further iterations of the protocols:

- **Assess the combination of the two protocols' design goals.** The protocol combines a “fixed-rate yield generation” with a “1:1 swap of similar assets across chains”, and the combination of these two goals added complexity to the codebase.
- **Reevaluate manager privileges and constraints.** Some of the system's complexity comes from the duality of having the manager as both a privileged actor, but restricted. For example most of the arithmetic complexity stems from the need of share tracking to prevent the manager from evaluating incorrect obligations, but if this assumption could

be mitigated differently, shares could be removed, and overall arithmetic could be simplified.

- **Reevaluate bridging assumptions and capacity.** While the system was originally designed to be bridge-independent, the coupling to Chainlink allowed a more robust design with additional safeguards.
- **Balance modularity vs simplicity.** Some code patterns, such as supporting multiple asset transfers, indicate that the code is built with modularity in mind, however, such modularity comes with complexity that can be prevented. Further design analysis on what needs to be modular versus what can follow one set of parameters could simplify the design.
- **Consider a more standard approach to asset transfers.** The reliance on TransferHelper, while decreasing the cost of asset transfers, creates a complex tracking of asset flow within the system. Some features (e.g. fees on CCIP) suffer from this complexity (e.g. the user can't use traditional allowance and needs to correctly set a refund threshold or lose part of the fees), and the system could be simplified to prevent mistakes.
- **Integrate an ERC4626 edge-case analysis.** Several issues were found alongside the integration of ERC4626

This report is part of a multiple stages review of the Stable Vault (1/4).

Security findings

ID	Title	Severity	Impact	Likelihood	Status
1	Dust positions can prevent adding a new subvault	Low	Low	Low	Acknowledged
2	allowOutOfOrderExecution = false is deprecated	Medium	Low	High	Fixed
3	ChainId collision risks on ccipReceive	Medium	High	Low	Fixed
4	Front-running in applyWithdrawalPolicy	Low	Low	Low	Fixed
5	Unstable rounding in growth factor	Informational	Informational	Low	No fix required
6	Temporary denial of service through funds move	Low	Low	Low	Acknowledged
7	Cross-chain message with dust might be trapped	Informational	Informational	Low	Fixed
8	The earning chain might send an outdated balance	Low	Low	Low	Partially fixed
9	ATokenVaultMerkleRewardClaimer's owner can drain the vault	Medium	High	Low	Fixed
10	Fee subsidy with stuck native currency token	Low	Low	Medium	Fixed
11	Lack of zero-address check on previous signer in transferSigner	Informational	Informational	Low	Acknowledged

12	Manager can drain dust through migration operations	<i>Low</i>	<i>Medium</i>	<i>Low</i>	<i>Acknowledged</i>
13	Lack of Sequencer's uptime check can lead to stale prices and balances being used	<i>High</i>	<i>High</i>	<i>Medium</i>	<i>Fixed</i>
14	Uncaught adapter reverts can block getChainBalance	<i>Low</i>	<i>Low</i>	<i>Low</i>	<i>Fixed</i>
15	Deposit share should be calculated from the net deposited amount	<i>Informational</i>	<i>Informational</i>	<i>Informational</i>	<i>Acknowledged</i>

Dust positions can prevent adding a new subvault

Impact: Low

Likelihood: Low

Status: Acknowledged

001

Summary

An attacker can intentionally create dust positions that become unmigratable, preventing deletion of the corresponding subvault.

Detailed description

The StableVault enforces a limit on active subvaults via MAX_ACTIVE_SUB_VAULTS.

```
function _validateAmountOfActiveSubVaults() internal view {  
    require($storage().activeSubVaultsIds.length <= MAX_ACTIVE_SUB_VAULTS,  
    TooManyActiveSubVaults());  
}
```

Figure 1: [StableVault.sol:492-494](#)

A subvault is considered active if it has any position. To remove a subvault, its underlying positions must either be withdrawn (by the user) or migrated (by the manager)

A position with a small number of shares might not be migratable to a subvault with a higher conversion rate, if the newly calculated shares round to zero:

```
uint256 userNewShares =  
userOldShares.rayMulDown(oldConversionRate).rayDivDown(newConversionRate);  
    // Do not allow the user position share quantity to deplete to zero which can  
    happen if a user has a small  
    // userOldShares quantity and newConversionRate is large.  
    require(userNewShares > 0, Errors.InvalidAmount());
```

Figure 2: [StableVault.sol:452-455](#)

An attacker can create a dust position by depositing a minimal amount (e.g., 1 wei). If this position is migrated to a subvault with a low conversion rate, it might not be possible to migrate it again. If this happens while the system reaches the MAX_ACTIVE_SUB_VAULTS number of vaults, then the system will have one subvault that will not be removable.

We classified this issue as low because it only affects one subvault slot.

Recommendations

Consider:

- Adding a minimum deposit amount
- Count “+1” subvault in the number of expected subvaults, and use a “trapping” vault for dust positions.

Status

Aave Labs stated:

Acknowledged. The MAX_ACTIVE_SUB_VAULTS cap is 201 which corresponds with a mapping maintained off-chain to ensure users are only migrated to the predefined 201 sub-vaults. Furthermore, if all positions are to urgently be removed from a sub-vault, then all users within can be migrated to the sub-vault with a per second rate of 1 RAY which would maintain a conversion rate of 1 RAY.

allowOutOfOrderExecution = false is deprecated

Impact: Low

Likelihood: High

Status: fixed

002

Summary

The CCIPAdapter uses `allowOutOfOrderExecution = false`, which is deprecated, and might create future incompatibility issues with CCIP.

Detailed description

CCIPAdapter uses `allowOutOfOrderExecution = false`:

```
extraArgs: Client._argsToBytes(  
    Client.GenericExtraArgsV2({gasLimit: bridgeParams.gasLimit,  
allowOutOfOrderExecution: false}))  
)
```

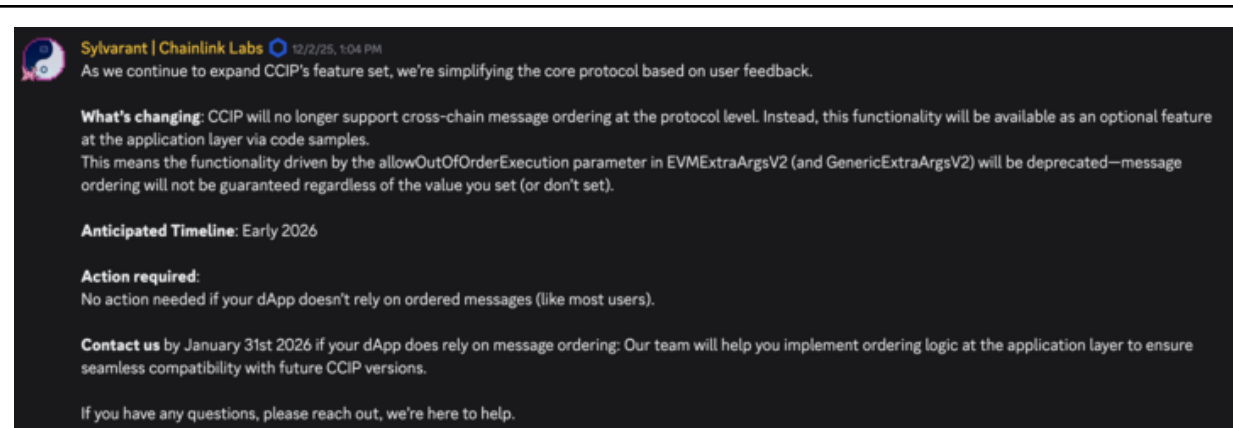
Figure 1: [CcipAdapter.sol#L104](#)

The CCIP documentation states that:

Note: Functionality for `allowOutOfOrderExecution = false` (ie., enforcing In-Order cross-chain messages) is being deprecated in early 2026.

Figure 2: <https://docs.chain.link/ccip/concepts/best-practices/evm#best-practices>

And their Discord mentions:



Sylvant | Chainlink Labs 12/2/25, 104 PM

As we continue to expand CCIP's feature set, we're simplifying the core protocol based on user feedback.

What's changing: CCIP will no longer support cross-chain message ordering at the protocol level. Instead, this functionality will be available as an optional feature at the application layer via code samples. This means the functionality driven by the `allowOutOfOrderExecution` parameter in `EVMExtraArgsV2` (and `GenericExtraArgsV2`) will be deprecated—message ordering will not be guaranteed regardless of the value you set (or don't set).

Anticipated Timeline: Early 2026

Action required:
No action needed if your dApp doesn't rely on ordered messages (like most users).

Contact us by January 31st 2026 if your dApp does rely on message ordering: Our team will help you implement ordering logic at the application layer to ensure seamless compatibility with future CCIP versions.

If you have any questions, please reach out, we're here to help.

Figure 3: Chainlink's discord

As a result this feature will be removed.

Note that even without this removal, `allowOutOfOrderExecution = false` does not fully protect against out-of-order messages, due to the manual retry mechanism.

Recommendations

Do not use `allowOutOfOrderExecution = false` in CCIPAdapter.

Status

Fixed with [PR148](#)

ChainId collision risks on ccipReceive

Impact: High

Likelihood: Low

Status: fixed

003

Summary

The lack of chainId <> sourceChainSelector validation can allow the chain adapter's address for chainID zero to be used for any non-set source chain.

Detailed description

ccipReceive uses two mappings, _chainSelectorOf and _chainIdOf:

- _chainSelectorOf maps a chainID to a chain selector
- _chainIdOf maps a chain selector to a chainID

```
mapping(uint256 chainId => uint64 ccipChainSelector) internal _chainSelectorOf;  
mapping(uint64 ccipChainSelector => uint256 chainId) internal _chainIdOf;
```

Figure 1: [CcipAdapter.sol#L28-L29](#)

Due to the way mapping works in Solidity, if a chain selector is not present in the _chainIdOf mapping, it will return the chainID zero (and similarly for _chainSelectorOf).

ccipReceive does not validate that a given message.sourceChainSelector is correctly set in _chainIdOf.

This can be abused to:

- Have the adapter address for chainID zero to be used for any source chain:

```
require(  
    abi.decode(message.sender, (address))  
    ==  
    _destinationChainAdapterOf[_chainIdOf[message.sourceChainSelector]],  
    OnlyDestinationChainAdapter()  
);
```

Figure 2: [CcipAdapter.sol#L126-L129](#)

- Generate incorrect events

```
emit TokenReceptionFailed(  
    message.messageId,
```

```
        _chainIdOf[message.sourceChainSelector],  
        message.destTokenAmounts[i].token,  
        message.destTokenAmounts[i].amount  
    );  
}  
emit BridgedFundsProcessingFailed(  
    message.messageId, _chainIdOf[message.sourceChainSelector],  
    abi.encode(message), err  
);
```

Figure 3: [CcipAdapter.sol#L139-L148](#)

Note: We classified this issue with a high severity due to the risk of the adapter's address for chainID zero to be used in any non-set source <> destination, but while the impact is severe, such a scenario would require a malicious or compromised initial deployment.

Recommendations

In ccipReceive, check that:

```
_chainSelectorOf[_chainIdOf[message.sourceChainSelector]] ==  
message.sourceChainSelector
```

Status

Fixed with [PR148](#)

Front-running in applyWithdrawalPolicy

Impact: Low

Likelihood: Low

Status: fixed

004

Summary

A front-running attack on signature validation can allow an attacker to prevent a user from calling `executeWithdrawal/exchangeIoUTokens`.

Detailed description

If `executeWithdrawal / exchangeIoUTokens` contains a signature, `applyWithdrawalPolicy` will use it and consume the signer's nonce:

```
function applyWithdrawalPolicy(WithdrawalRequest calldata request) external
override returns (uint256) {
    (uint256 amountOutRay, address signer, uint256 nonce) =
    _previewWithdrawalPolicy(request);
    if (signer != address(0)) {
        _markNonceAsUsed(signer, nonce);
    }
}
```

Figure 1: [WithdrawalPolicy.sol#L111-L115](#)

Anyone can call `applyWithdrawalPolicy`. As a result, an attacker can front-run a user calling `executeWithdrawal / exchangeIoUTokens` and consume the corresponding signature/nonce.

Recommendations

Add an access control to `applyWithdrawalPolicy` so that its calls can only originate from the gateway and Stable vault contracts.

Status

Fixed with [PR134](#)

Unstable rounding in growth factor

Impact: Informational

Likelihood: Low

Status: No fix required

005

Summary

The `rpow` function used to compute the growth factor uses the “round to nearest” strategy, which can lead its value to favor the user slightly.

Detailed description

`_previewSubVaultConversionRate` uses `rpow` from `Solady` when computing the growth factor:

```
uint256 growthFactor =
$storage().subVaultById[subVaultId].perSecondRate.rpow(secondsSinceLastAccrual);
    // The conversion rate is used across different operations (e.g.
converting assets to shares on deposits,
    // converting shares to assets on withdrawals, computing total vault
obligations, etc.).
    // The conversion rate calculation rounds down to ensure a conservative
and consistent value that subsequent
    // operations can then use to apply their context-specific rounding on
top with the intention of favoring
    // the protocol.
newConversionRate = newConversionRate.rayMulDown(growthFactor);
```

Figure 1: [StableVault.sol#L500-L506](#)

`Rpow` uses the “round to nearest” strategy:

```
let zxRound := add(zx, half) // Round to the nearest number.
```

Figure 2: [FixedPointMathLib.sol#L761](#)

As a result, `growthFactor` might round up, favoring the user.

`growthFactor` is then multiplied by the new conversion rate, using a rounding down strategy, which might limit the risk.

Status

We continued the exploration of the risks after reporting the issue in [Appendix C](#). No code changes are required.

Temporary denial of service through funds move

Impact: Low

Likelihood: Low

Status: Acknowledged

006

Summary

One of the system's assumptions relies on users being able to withdraw their funds at any time. To do so, the user needs to have IOU tokens on the same chain as the system's liquidity.

A malicious manager could constantly move the funds between the chains (e.g. from the moment the funds arrive from the bridge, bridging them again), such that the funds are constantly in transit, and the users cannot exit.

The issue has a low severity, given that no funds will be lost, and the attack would stop after an admin intervention.

Recommendations

This issue can be addressed by either:

- Adding a time buffer between the sending of funds, to prevent the manager from constantly sending the funds back and forth
- Documenting the risks, and relying on the admin to remove the manager's rights/permissions if such a scenario were to occur

Status

The Aave Labs stated:

Acknowledge, won't fix. We treat this as an admin/trust-model risk and document it as an accepted scenario mitigated via operational monitoring and removing the manager if abuse is detected.

Cross-chain message with dust might be trapped

Impact: Informational

Likelihood: Low

Status: Fixed

007

Summary

A rounding protection on deposits might lead some messages not being processed if they arrive with a tiny amount of tokens.

Detailed description

When a message is processed, the tokens might be directly deposited into a strategy:

```
function _depositToStrategy(address asset, uint256 amount, address strategy)
internal {
    require(amount > 0, Errors.ZeroAmount());
    require($storage().strategyConfigs[strategy].depositAllowed,
DepositsToStrategyDisabled(strategy));
    IERC20(asset).forceApprove(strategy, amount);
    (bool callSucceeded, bytes memory data) =
        strategy.call(abi.encodeCall(IERC4626.deposit, (amount, address(this))));
    require(callSucceeded, DepositIntoStrategyFailed(strategy));
    uint256 sharesMinted = abi.decode(data, (uint256));
    require(sharesMinted != 0, Errors.InsufficientAmountOut());
    emit AssetAllocated(asset, strategy, amount);
}
```

Figure 1: [Allocator.sol#L378](#)

If the amount of tokens deposited is small, the number of shares might round to zero. If that happens, the transaction reverts (`sharesMinted != 0`).

This can cause some assets to never be processed.

The issue is informational, given that the data part of a message can still be processed, even if the attached tokens message reverts, due to the non-reverting structure of `ccipReceive`.

Recommendations

Consider either:

- Call `previewDeposit` for small amounts, and skip if the result is zero
- Document the risks, and ensure `ccipReceive` never blocks on asset reception

Status

Fixed in commit [6efc45f](#) and commit [74a03a4](#) by adding `tryDepositToStrategy`, which does not revert in case of failed deposit.

The earning chain might send an outdated balance

Impact: Low

Likelihood: Low

Status: Partially fixed

008

Summary

Two undocumented attack vectors can cause the earning chain to update the accounting chain snapshot with an out-of-date balance.

Detailed description

Reentrancy on rebalance

During a swap operation, the manager can control the `swap.swapper` contract, to which `executeSwap` is called:

```
function _swap(SwapParams memory swap) internal {  
    ...  
    // Transfer assetIn to the swapper  
    IERC20(swap.assetIn).safeTransfer(swap.swapper, swap.amountIn);  
  
    // Execute the swap and require 1:1 conversion  
    uint256 amountOut = ISwapper(swap.swapper).executeSwap(swap.assetIn,  
swap.assetOut, swap.amountIn, swap.data);
```

Figure 1: [Allocator.sol#L301-L310](#)

During the execution of `executeSwap`, the allocator's balance is decreased. If the swapper contract uses `_getTotalAssetBalance`, it will therefore return a lower value than the system's expected value for the allocator.

If a balance update is triggered at that moment, the earning chain will send an incorrect balance to the accounting chain.

By re-entering directly on swap, a malicious manager can chain together multiple transfers, and a balance update, such that the earning chain sends a balance update where all its assets are at zero to the accounting chain.

Timing attack on update

When the accounting chain sends funds to the earning chain, it first updates its local view of the earning chain account for the asset sent:

```
// Increment the chain balance snapshot for the target chain.  
_updateChainBalanceBeforeBridging(chainId, amount.assetDecimalsToRay(asset));  
IAccountingChainGateway(GATEWAY).sendPushFundsToChainMessage(asset, amount,  
chainId, bridgeParams);
```

Figure 2: [FundsHandler.sol#L159-161](#)

However if a message is sent from the earning chain, after this update, but before the funds are received on the earning chain, the update will not include the funds sent. This will make the accounting chain update its view of the assets with an outdated balance.

Both scenarios lead the accounting chain to see fewer assets than exist in the system. This will make $\text{obligation} > \text{asset}$ at the accounting chain level, blocking the withdrawal of interest (but not principal), until a new balance snapshot is taken.

Recommendations

To fix the timing attack, consider removing the call to `_updateChainBalanceBeforeBridging`. However the downside is that it can temporarily disable interest withdrawal, until the earning chain sends its update.

The reentrancy issue requires further consideration to be fixed properly: a `nonReentrant` modifier would reduce the attack scope (as only one asset could be temporarily set to zero). A cross-contract lock would probably add too much complexity for the benefits.

For a broader consideration:

- Document that $\text{obligation} > \text{asset}$ can temporarily occur from the accounting chain's point of view.
- Ensure monitoring and any future circuit breaker do not rely on that invariant.

Status

Partially fixed. The timing attack was fixed in [PR145](#), but the reentrancy vector was still open.

ATokenVaultMerkleRewardClaimer's owner can drain the vault

Impact: High

Likelihood: Low

Status: Fixed

009

Summary

The lack of filtering for aToken transfer in `claimMerkleRewards` allows the contract's owner to drain the vault.

Detailed description

`claimMerkleRewards` allows the owner to transfer any token for which the contract's balance was increased through `merkleDistributor.claim`:

```
function claimMerkleRewards(
    address[] calldata tokens,
    uint256[] calldata amounts,
    bytes32[][] calldata proofs,
    address[] calldata rewardTokensToForward,
    address destination
)
public
override
onlyOwner
{
    require(_s.merkleDistributor != address(0), "MERKL_DISTRIBUTOR_NOT_SET");
    require(tokens.length == amounts.length && tokens.length == proofs.length,
"ARRAY_LENGTH_MISMATCH");

    uint256[] memory currentBalancesOfRewardTokens = new
uint256[] (rewardTokensToForward.length);
    for (uint256 i = 0; i < rewardTokensToForward.length; i++) {
        currentBalancesOfRewardTokens[i] =
IERC20Upgradeable(rewardTokensToForward[i]).balanceOf(address(this));
    }

    address[] memory users = new address[] (tokens.length);
    for (uint256 i = 0; i < tokens.length; i++) {
```

```

        users[i] = address(this);
    }
    IMerkleDistributor(_s.merkleDistributor).claim(users, tokens, amounts, proofs);
    emit MerkleRewardsClaimed(_s.merkleDistributor, tokens, amounts);

    for (uint256 i = 0; i < rewardTokensToForward.length; i++) {
        uint256 newBalance =
IERC20Upgradeable(rewardTokensToForward[i]).balanceOf(address(this));
        uint256 amountToForward = newBalance - currentBalancesOfRewardTokens[i];
        if (amountToForward > 0) {
            IERC20Upgradeable(rewardTokensToForward[i]).safeTransfer(destination,
amountToForward);
            emit MerkleRewardsTokenForwarded(rewardTokensToForward[i],
destination, amountToForward);
        }
    }
}

```

Figure 1: [ATokenVaultMerkleRewardClaimer.sol#L32-L66](#)

This includes aToken. A malicious owner can abuse this function by setting merkleDistributor to a malicious contract, instead of a valid distributor, and re-enter into the vault to deposit aToken (or underlying that would be converted to aToken). The deposited aToken would then be sent back to the vault's owner, while vault shares are minted.

The owner can then withdraw these shares and drain the vault's funds.

For example:

- Eve is a malicious owner
- She sets merkleDistributor to a controller contract
- She calls claimMerkleRewards with rewardTokensToForward = [ATOKEN]
- During the call to merkleDistributor.claim, Eve re-enters into the aTokenVault, and calls depositATokens
- Then newBalance - currentBalancesOfRewardTokens will be equal to the amount of tokens deposited
- As a result the aToken deposited will be sent back to Eve
- Finally, Eve withdraws the minted shares, and effectively drains the vault

Recommendations

Prevent ATOKEN from being used in rewardTokensToForward

Through our discussion with Aave Labs, we also recommended to consider simplifying the integration of the Angle protocol and use a simpler approach to collect the reward by either

- Only call the *claim* function
- Or using the *toggleOperator* function

Remove the tokens transfer in `claimMerkleRewards`, and instead use `emergencyRescue` to transfer tokens.

This would reduce code complexity and enable access controls where a non-owner account could claim the reward.

Status

Fixed with commit [67395a8ac24422b0de53a3703b5409081b8576a3](#)

Fee subsidy with stuck native currency token			
<i>Impact: Low</i>	<i>Likelihood: Medium</i>	<i>Status: fixed</i>	<i>010</i>

Summary

The lack of `msg.value` validation allows the reuse of the native tokens left in the CCIPAdapter contract to pay for the CCIP fees.

Detailed description

When sending the CCIP fee to the CCIP router, there is no check to ensure that the fee amount was actually sent by the caller:

```

uint256 estimatedFeeAmount = IRouterClient(CCIP_ROUTER).getFee(chainSelector,
message);
uint256 msgValue;
if (feeToken == Constants.NATIVE_CURRENCY) {
    msgValue = estimatedFeeAmount;
}
if (allocatedFeeAmount > estimatedFeeAmount) {
    uint256 excessFee = allocatedFeeAmount - estimatedFeeAmount;
    if (excessFee > feeRefundThreshold) {
        _triggerFeeRefund(feePayer, feeToken, excessFee);
    }
}

```

Figure 1: [CcipAdapter.sol#L176-L187](#)

As a result, if the fee is paid by the native currency, the caller can set `feeAmount` to zero, and use instead any native currency leftover from the adapter.

Recommendations

Add `require(allocatedFeeAmount >= estimatedFeeAmount)` in `_sendMessageWithFeePayer`

Status

Fixed with [PR155](#)

Lack of zero-address check on previous signer in transferSigner			
Impact: Informational	Likelihood: Low	Status: Acknowledged	011

Summary

transferSigner should check for the zero address on previous signer:

```
function transferSigner(uint32 entityId, address previousSigner, address
newSigner) external {
    _removeSigner(entityId, previousSigner);
    _addSigner(entityId, newSigner);
    emit SignerTransferred(msg.sender, entityId, newSigner, previousSigner);
}
```

Figure 1: transferSigner function

Otherwise, a module could be misconfigured, because:

- _removeSigner will be a no-op operation
- And the signer1 will be set to the new signer

This will result in the module having signer1 set but not signer2.

Recommendations

Add require(previousSigner !=0) in transferSigner

Status

Aave Labs stated:

Acknowledged. In the event a signer is added to the validation module for a given account before it is installed onto the account, the account may call the validation module to add another signer. Once both signer slots are non-address(0) addresses, the account may proceed to call MultiSigValidationModule::onUninstall to reset the signers for the account back to address(0).

Manager can drain dust through migration operations

Impact: Medium

Likelihood: Low

Status: Acknowledged

012

Summary

The manager can abuse the rounding happening in the user migration operation to steal dust from the Stable vault. This includes potentially draining small amounts that are related to the users' principals.

Detailed description

When a user is migrated to a new subvault, the new shares are rounded down:

```
function _migrateUserToSubVault(address user, uint256 oldSubVaultId, uint256
newSubVaultId) internal {
    uint256 oldConversionRate = _accrueSubVaultConversionRate(oldSubVaultId);
    uint256 newConversionRate = _accrueSubVaultConversionRate(newSubVaultId);
    uint256 userOldShares = $storage().positions[user].shares;
    // Round down the amount of shares after sub-vault migration, so that the
    rounding is in favor of the protocol.
    uint256 userNewShares =
userOldShares.rayMulDown(oldConversionRate).rayDivDown(newConversionRate);
    // Do not allow the user position share quantity to deplete to zero which can
    happen if a user has a small
    // userOldShares quantity and newConversionRate is large.
```

Figure 1: _migrateUserToSubVault functions

While this rounding ensures that no shares are created for free, it allows a manager to slowly reduce the user's underlying value. In particular the manager can migrate back and forth a user between two subvaults, which will make the user's shares decrease more and more.

By doing so, the manager will slowly reduce the total shares tracked within the system, reducing the vault obligation, which allows the manager to withdraw the leftover by calling `claimSurplusInterest`

Note: we classify this issue as medium severity instead of high severity, given that while the funds can be drained and one of the core invariants is broken (user principal can be stolen), the actual amounts being affected are low

Recommendations

Fixing this issue while keeping the migration operation is not straightforward without significant code change.

Due to the limited impact of the issue, partial mitigation strategies can be considered, such as having a rate limit on how often a user can be migrated.

Status

Aave Labs stated:

Acknowledged. While repeated migrations can theoretically shave a bounded amount per hop due to rounding, the impact is economically negligible: for any realistic position size, normal interest accrual outpaces the rounding loss by many orders of magnitude. Achieving even 1 wei of asset-level loss would require an impractical number of back-and-forth migrations (and prohibitive gas/block usage).

Lack of Sequencer's uptime check can lead to stale prices and balances being used

Impact: High

Likelihood: Medium

Status: Fixed

013

Summary

The staleness check for the price/balance Oracles relies on `block.timestamp`, which is not robust if the L2 sequencer is down.

Detailed description

`Block.timestamp` is used to check for staleness:

```
if (
    bundleTimestamp < block.timestamp && block.timestamp - bundleTimestamp >=
HEARTBEAT + PUBLISH_BUFFER_SECONDS
) {
    isStale = true;
}
```

Figure 1: *ChainlinkChainBalanceOracleAdapter.sol#L59-L63*

On L2, timestamps are not always reliable. For example, on Arbitrum if the sequencer is down, `block.timestamp` will be the timestamp when the message was added to the delayed inbox:

Furthermore, for transactions that are force-included from the parent chain (bypassing the Sequencer), the block timestamp will be equal to either the parent chain timestamp when the transaction was put in the Delayed Inbox on the parent chain (not when it was force-included), or the child chain timestamp of the previous child chain block, whichever of the two timestamps is greater.

Figure 2: [Arbitrum's documentation](#)

The timestamp on L2 can also be in the future (up to 1 hour on Arbitrum). If the following happens:

- The L2's timestamp is 1 hour in the future
- The sequencer has an issue and stops producing blocks

Then all the messages included in the delayed inbox for 1 hour (L1's time) will have the same `block.timestamp`.

As a result, the staleness check will not work, and the Stable vault is likely to use a stale price or balance.

Recommendations

Use Chainlink's sequencer feeds: <https://docs.chain.link/data-feeds/l2-sequencer-feeds> to check whether the Oracle is not stale.

Status

The recommendation was implemented in [PR 177](#).

Uncaught adapter reverts can block getChainBalance

Impact: Low

Likelihood: Low

Status: Fixed

014

Summary

ChainBalanceOracleAdapter.getChainBalance has several conditions that could cause it to revert:

```
IEarningChainStateProvider.State memory state = abi.decode(bundle,
(IEarningChainStateProvider.State));
require(state.version == SCHEMA_VERSION,
InvalidEarningChainStateVersion(SCHEMA_VERSION, state.version));
EarningChainStateSchemaV1.BalanceSnapshot memory balanceSnapshot =
abi.decode(state.data, (EarningChainStateSchemaV1.BalanceSnapshot));
require(
balanceSnapshot.chainId == EARNING_CHAIN_ID,
InvalidSnapshotChainId(EARNING_CHAIN_ID, balanceSnapshot.chainId)
);
```

Figure 1: ChainlinkChainBalanceOracleAdapter.sol#L65-L74

If a revert happens, ChainBalanceOracle.getChainBalance will revert:

```
function getChainBalance(uint256 chainId) external view override returns
(uint256) {
require($storage().oracleAdapterByChainId[chainId] != address(0),
ChainBalanceOracleAdapterNotFound(chainId));
IChainBalanceOracleAdapter.OracleResponse memory response =
IChainBalanceOracleAdapter($storage().oracleAdapterByChainId[chainId]).getChainBalanc
e(chainId);
return response.isStale ? 0 : response.balanceRay;
}
```

Figure 2: ChainBalanceOracle.sol#L53-L59

This can cause the underlying operations (ex: withdrawal) to be blocked.

Recommendations

Surround the call to `ChainBalanceOracleAdapter.getChainBalance` with a try/catch

Status

The recommendation was implemented in [PR173](#)

Deposit share should be calculated from the net deposited amount

Impact: Informational

Likelihood: Low

Status: Acknowledged

015

Summary

When calculating the shares, deposit uses the original amount deposited instead of the net amount deposited:

```
uint256 netDepositAmount = IFundsHandler(FUNDS_HANDLER).processDeposit(asset,
amount);

// Calculate the number of shares to mint based on the full amount deposited.
// If (amount - netDepositAmount) > 0, then this ~amount will be treated as
interest earned.
// Round down the division to undershoot the amount of granted shares,
favoring the protocol.
uint256 shares = amount.assetDecimalsToRay(asset).rayDivDown(conversionRate);
```

Figure 1: StableVault.sol#L223-225

The difference might come from slippage induced by rounding, or withdrawal fees, depending on the underlying vault. The assumption is that the difference between the net deposit and amount will be treated as interest, however they might represent assets that are not held by the system (ex: withdrawal fee). As a result, the interest earned by the depositor is taken from the system's other users.

Note: we classified this issue as informational as the impact of such a slippage would be minimal, unless a non-standard vault would be added to the system

Recommendations

Use netDepositAmount instead of amount when computing the shares.

Status

Aave Labs stated:

Acknowledge, won't fix. The team intends to allow for users to withdraw the entirety of original deposits which is determined by the net deposit amount. Given a default strategy

is not address(0), the slippage amount incurred on deposit contributes to the user's interest earned; this amount would not be allowed to be withdrawn if the system's assets do not sufficiently cover global original deposits.

Appendix A: Design recommendations against depegging

The protocol's lack of price oracles and its 1:1 fixed exchange rate between assets create a depeg risk. If any asset's price moves from \$1, an attacker can deposit the depegged asset and withdraw a healthy asset, extracting value. For example: deposit 1000 asset A (worth \$10) and withdraw 1000 asset B (worth \$1000).

Even with an oracle, additional risks exist:

- A malicious manager could donate unhealthy assets to the system and withdraw healthy assets via `claimSurplusInterest`
- Donation attacks where an attacker accumulates depegged assets and donates them to the Allocator, allowing withdrawal of principal plus interest while potentially depreciating other users' original deposits.

The core issue is combining a “fixed rate yield generation” protocol with a “1:1 swap of similar assets across chains” protocol. Fixed yield generation is achievable, but robust 1:1 swaps are more challenging to build.

These risks were known and documented by Aave Labs. The following contains two proposed solutions to address the problem. Both directions require further investigation and may have unaddressed problems.

Solution 1: Liquidity Pool Per Asset Design

Approach

Redesign the system to use separate liquidity pools per asset:

- Users deposit assets into asset-specific liquidity pools.
- Managers can pull from these pools.
- Implement an IOU system per token (instead of a global IOU system).
- Use oracles to exchange IOUs for assets when not directly available in the original liquidity pool.

Example User Flow

Bob makes a withdrawal request for token A, receives 100 IOU_A tokens, bridges them to another chain, and exchanges 100 IOU_A for 100 token B using an oracle to verify the swap price.

Caveat

There is a risk that liquidity may not be located on a chain where the user can exchange their IOU. For example, if liquidity is in token C, the system needs an oracle A-C on that chain, otherwise the user cannot withdraw. This requires system configuration to ensure correct paths between liquidity and available swaps, or a mechanism to exchange IOU_A for IOU_B (which adds complexity).

Solution 2: Oracle-Based Asset Valuation with Current Architecture

Approach

Keep the current architecture but couple all asset valuation to an oracle:

- On deposit: users do not receive $\text{amount} / \text{conversionRate}$ directly. Instead, the amount is first adjusted based on the token's price via oracle.
- On balance computation: when the Allocator computes its balance, the same adjustment applies: the Allocator balance is adjusted based on current token prices.

So that:

- Obligations and shares are computed based on the underlying asset price at the time of their creation.
- Current asset amounts are computed based on the current asset market value.

This approach may mitigate different attack vectors:

- If an asset depegs and a user deposits it, the user receives fewer shares.
- Donation attacks would increase asset valuation properly without inflating obligations.

Example User Flow

Alice deposits 1000 USDC (worth \$950 due to depeg), receives shares based on the oracle-adjusted \$950 value, and later withdraws USDT where the output amount is calculated using the current oracle price of USDT.

Caveat

An unexplored side effect to consider is the proper handling of an asset whose price goes above \$1, and potential side effects of arbitrage opportunity.

Appendix B: Defense in depth recommendations

The following contains general recommendations to increase the system's robustness while not being associated with a specific finding:

- **Use a single asset in `publishMessageToChainWithFeePayer` instead of an array.** In practice the function is only called with 1 asset. Moreover Chainlink has limited support for multiple-asset transfers
- **Have a registry of swapper contracts.** This will limit the risk of malicious action from the owner.
- **Clarify the handling of native currency.** In particular in the case of double-entry tokens, such as Celo, which the integration could skew the internal account if it was accepted as both native currency and ERC20 tokens.
- **Ensure the earning chain supports EIP 1153.** [PR136](#) makes the code rely on transient operations.
- **Consider splitting `Gateway.receiveMessage` into `Gateway.receiveData` and `Gateway.receiveFunds`.** This will clarify the calls and related paths
- **Consider the Oracle to be stale if `price <= 0` and `balance < 0` in `ChainlinkPriceOracleAdapter` and `ChainlinkChainBalanceOracleAdapter`.** Chainlink Oracle could in theory return a negative integer, which would be incorrectly interpreted by the adapter
- **Document that the transfer operation will decrease the user's guarantee principal.** This behavior might be unexpected and create confusion among users

Appendix C: Rounding analysis

The following contains the analysis done on the rounding related to the conversion rate:

$$\text{conversionRate} = \text{conversionRate} * \text{perSecondRate}^{\text{seconds}}$$

Precision loss worst case scenario

The conversion rate is monotonically non-decreasing for both the rate and the seconds passed. We will use that assumption to evaluate the maximum loss of precision in the worst case scenario. Our analysis shows it is negligible.

Given the system parameters:

- conversionRate starts at 1 RAY
- perSecondRate is between 1 and 1.000000005781378656804591713

rpow uses the exponentiation by squaring algorithm, and therefore its loss of precision can grow on every loop iteration, which is equal to the number of bits of secondsSinceLastAccrual.

A safe bound for a worst case scenario is secondsSinceLastAccrual = 2^{29} , which is above 17 years of usage.

We compare two rpow implementations based on these parameters:

- One always rounding down
- One always rounding up

Such analysis creates an upper bound on the deviation that can happen for a nearest rounding, considering that the nearest rounding can at worst, be equal to either always rounding down or up. Note that this is an over-approximation, given that in practice the nearest rounding strategy will not always round up or down at all iterations.

For such a scenario, we found:

- up = 22.283667769544459100500199793
- near = 22.283667769544459089669968047
- down = 22.283667769544459088537757808
- real = 22.283667769544459092960868570 (computed through [wolframalpha](https://www.wolframalpha.com))

The difference is below 10^{-16} . Given that this scenario is a safe upper bound, we can conclude that at most the loss of precision for all three strategies is negligible.

Note: this analysis did not include splitting rpow into individual mulDown operations every second, however we believe the impact of such operations would be minimal, and not realistic

Reuse problem

The current implementation suffers from the [rounding reuse problem](#), where the same variable (conversionRate) is used in places where it should round down (e.g. to compute new shares), or up (e.g. to compute obligations) to favor the protocol.

A previous implementation used an approach where the multiplication was either rounding down or up: [cce4796](#).

The solution was removed due to the complexity risk. Nonetheless this implementation was incomplete, as it did not consider the rounding happening in rpow, which is non-stable, leading the conversion rate result to either overall be overestimated or underestimated.

A perfect implementation would therefore:

- Use a rpow and multiplication that always round up for operations that need to over-estimate the rate
- Use a rpow and multiplication that always round down for operations that need to under-estimate the rate

However given the additional gas cost (as two conversionRate would need to be stored), and the added complexity, such a dual version of the conversion rate is not required. In particular due to the low deviation demonstrated in the previous section.

Splitting strategy to break the rate growth ordering

The loss of precision within rpow, and the mix of two rounding strategies (nearest for rpow and rounding down for the multiplication) leads to breaking the product law of exponents, such as:

$$perSecondRate^{t_0 + t_1} \neq perSecondRate^{t_0} * perSecondRate^{t_1}$$

As a result, by splitting a rate's growth into either smaller steps, or one larger step, an attacker can force the conversion rate to grow slightly faster or slower.

We performed several fuzzing campaigns to find cases where this could lead a conversion rate with a lower rate per-second to grow faster than a conversion rate with a high rate per-second, such as:

$$perSecondRate0 < perSecondRate1 \Rightarrow conversionRate0 > conversionRate1$$

However, within the system's bounds and parameters, such a scenario was not found to be possible.

The reason is likely that while the growth might diverge upward or downward, the divergence amount is smaller than the expected divergence between the conversion rate growth.